TECHVISION
TECHNOLOGIES
IT Training & Placement

Java Full Notes

Java Programming Language – Introduction, History & Purpose

What is Java?

Java is a **high-level, class-based, object-oriented programming language** designed to have as few implementation dependencies as possible. It is:

- Simple
- Secure
- Portable
- Platform-independent
- Robust
- Multi-threaded
- Architecture-neutral

Java follows the principle of **"Write Once, Run Anywhere" (WORA)**, meaning code written in Java can run on any platform that supports Java without the need for recompilation.

Who Developed Java?

- **Creator:** James Gosling
- **Company:** Sun Microsystems
- **Year:** Development began in **1991**, first public release in **1995**
- James Gosling is known as the "**Father of Java**"

Original Name of Java:

- Java was originally called **Oak**.

- The name was changed to **Java** in 1995 because “Oak” was already a registered trademark.
- The name "Java" was inspired by **Java coffee** (from the Indonesian island of Java).

Why Was Java Developed?

Java was created with the following goals:

Purpose	Description
Platform Independence	Write Once, Run Anywhere
Security	Built-in security features (no pointers, sandbox environment)
Simplicity	Easier to learn than C++, avoids complex features like pointers
Object-Oriented	Supports modular, reusable, and organized code
Robust and Error-Free	Strong memory management and exception handling
Network Capable	Built-in support for network and distributed computing

Short History / Timeline of Java:

Year	Event
1991	Java project initiated by James Gosling (originally named Oak)
1995	First public version of Java released
1998	Java 2 (J2SE) released – more powerful and modular
2006	Sun Microsystems made Java open-source
2009	Oracle Corporation acquired Sun Microsystems
2014–2023	Java versions 8 to 21 released (Java 8 is still widely used)

Where is Java Used?

Java is used in a wide range of applications:

- Android App Development
- Web Applications (e.g., using Spring, JSP, Servlets)
- Enterprise Applications (e.g., ERP, Banking systems)
- Scientific and Research-based Applications
- Desktop GUI Applications (e.g., Swing, JavaFX)
- Embedded Systems
- Game Development
- Big Data and Cloud-based Applications

Java – First Program: Hello World

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
```

```
}
```

Line-by-Line Explanation:

1. `public class HelloWorld {`

- **`public`** – Access modifier: this class is accessible from anywhere.
- **`class`** – This keyword is used to define a class.
- **`HelloWorld`** – The name of the class (should match the file name `HelloWorld.java`).

Every Java program must have a class, and the file name must match the class name.

2. `public static void main(String[] args) {`

This is the **main method** — the starting point of any Java program.

- **`public`** – The method can be accessed from outside the class.
- **`static`** – Can be run without creating an object of the class.
- **`void`** – This method does not return any value.
- **`main`** – The name of the method where program execution begins.
- **`String[] args`** – An array that can hold command-line arguments.

Java always looks for the `main()` method to start executing the program.

3. `System.out.println("Hello, World!");`

This line prints the message on the console:

- **`System`** – A built-in class from the `java.lang` package.
- **`out`** – A static object of the `PrintStream` class, connected to the console.
- **`println()`** – Method used to print the string with a newline.

This line will output: **Hello, World!**

4. `Curly Braces {}`

These curly braces close the `main()` method and the `HelloWorld` class.

How This Program Works – Step-by-Step

1. **Write Code** in a file named `HelloWorld.java`.
2. **Compile** using:

   ```
   javac HelloWorld.java
   ```

   - The Java compiler converts it into **bytecode**.
   - It creates a file: `HelloWorld.class`.
3. **Run** using:

   ```
   java HelloWorld
   ```

   - This uses the **JVM (Java Virtual Machine)** to execute the `.class` file.

- JVM reads the bytecode and displays output.

Java is **compiled and interpreted** — it compiles to bytecode and then is interpreted by the JVM.

Output:

```
Hello, World!
```

What is a Variable in Java?

A **variable** is a name given to a **memory location** where data is stored.
Think of it like a **box** that holds some value — a number, word, or any information.

Example:

```
int age = 20;
String name = "Neeraj";
```

Here:

- `age` is a variable that stores a number `20`
- `name` is a variable that stores the text `"Neeraj"`

How to Declare a Variable

Syntax:

```
dataType variableName = value;
```

Example:

```
int marks = 90;
float pi = 3.14f;
char grade = 'A';
```

Rules for Naming Variables

1. **Only letters (a-z, A-Z), digits (0-9), _, and $ allowed**
   - Valid: `marks`, `roll_no`, `amount$`, `total1`
   - Invalid: `my-name`
2. **Must not start with a digit**
   - `1age` — wrong
   - `age1` — correct
3. **Java is case-sensitive**
   - `Age` and `age` are different variables
4. **Don't use Java keywords (like `class`, `int`, `static`)**
   - `int class = 5;` — wrong
5. **Use meaningful names**
   - `a = 10;` — unclear
   - `marks = 10;` — clear

Types of Data in Java (Data Types)

Java is a **statically typed language**, which means you must define the **type of data** a variable will store.

1. Java Primitive Data Types – Table with Size and Range

Data Type	Size (in bytes)	Value Range	Example
byte	1 byte (8 bits)	-128 to 127	`byte b = 100;`
short	2 bytes (16 bits)	-32,768 to 32,767	`short s = 15000;`
int	4 bytes (32 bits)	-2,147,483,648 to 2,147,483,647	`int age = 25;`
long	8 bytes (64 bits)	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	`long population = 10000000000L;`
float	4 bytes (32 bits)	~±3.4e−038 to ±3.4e+038 (7 digits precision)	`float pi = 3.14f;`
double	8 bytes (64 bits)	~±1.7e−308 to ±1.7e+308 (15 digits precision)	`double price = 99.99;`
char	2 bytes (16 bits)	Unicode characters (0 to 65,535)	`char grade = 'A';`
boolean	1 bit (JVM uses 1 byte internally)	true or false	`boolean isOn = true;`

2. Non-Primitive Data Types (Also called Reference Data Types)

Data Type	Description	Example
String	Sequence of characters (text)	`String name = "Neeraj";`
Array	Group of values	`int[] marks = {90, 80, 70};`
Class	User-defined complex type	`Student s = new Student();`

Simple Java Program using Variables & Data Types

```
public class Main {
    public static void main(String[] args) {
        int age = 20;
        String name = "Neeraj";
        boolean isStudent = true;
        char grade = 'A';

        System.out.println(name + " is " + age + " years
old.");
```

```
        System.out.println("Student: " + isStudent);
        System.out.println("Grade: " + grade);
    }
}
```

Getting User Input in Java

In Java, to take input from the user, we typically use the `Scanner` class from the `java.util` package. Here's a step-by-step guide on how to do it.

Steps to Get User Input:

1. **Import the Scanner class**:
   - This is used to take input from the user.
2. **Create an object of Scanner class**:
   - This object will be used to read different types of inputs.
3. **Use the appropriate method**:
   - For example, `nextInt()` for integer input, `nextLine()` for strings, etc.

Example Code (Taking User Input)

```
import java.util.Scanner;

public class UserInputExample {
    public static void main(String[] args) {
        // Create a Scanner object
        Scanner scanner = new Scanner(System.in);

        // Taking user input for an integer
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        // Taking user input for a string
        System.out.print("Enter your name: ");
        scanner.nextLine();  // To consume the leftover
newline character
        String name = scanner.nextLine();

        // Displaying the input
        System.out.println("Hello, " + name + "! You are " +
age + " years old.");

        // Close the scanner
        scanner.close();
    }
}
```

Explanation:

1. **`nextInt()`** – Reads an integer input.
2. **`nextLine()`** – Reads a string input. After using `nextInt()`, we use `scanner.nextLine()` to consume the leftover newline character, which is important before reading a string.

3. `scanner.close()` – Closes the scanner to avoid resource leaks (good practice).

Practice Questions for Beginners

Question 1: Taking Two Numbers as Input

Write a Java program that takes **two numbers** as input from the user, adds them, and prints the result.

Expected Output:

```
Enter first number: 10
Enter second number: 20
Sum is: 30
```

Question 2: Taking User's Name and Age

Write a program that asks the user to input their **name** and **age**. Then, display a greeting message.

Expected Output:

```
Enter your name: Neeraj
Enter your age: 23
Hello, Neeraj! You are 23 years old.
```

Question 3: Temperature Conversion

Write a program that takes **temperature in Celsius** as input from the user and converts it to **Fahrenheit**.

Formula:

```
Fahrenheit = (Celsius * 9/5) + 32
```

Expected Output:

```
Enter temperature in Celsius: 30
Temperature in Fahrenheit: 86.0
```

Question 4: Area of a Circle

Write a program that takes the **radius** of a circle as input from the user and calculates the **area** of the circle.

Formula:

```
Area = π * radius^2
```

Expected Output:

```
Enter radius: 5
Area of the circle: 78.54
```

What are Operators in Java?

Operators are special symbols used to perform **operations** on variables and values.

Example:

```
int a = 10 + 5;  // '+' is an operator
```

Types of Operators in Java

Java provides several types of operators. Here's a complete list:

1. Arithmetic Operators:

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	6 / 3	2
%	Modulus (Remainder)	5 % 2	1

2. Relational (Comparison) Operators

Used to compare two values. Returns `true` or `false`.

Operator	Meaning	Example	Result
==	Equal to	5 == 5	true
!=	Not equal to	5 != 3	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
>=	Greater than or equal	5 >= 5	true
<=	Less than or equal	5 <= 4	false

3. Logical Operators

Used to combine multiple conditions.

Operator	Meaning	Example	Result
`&&`	Logical AND	`(5 > 3 && 4 > 2)`	true
`||`	Logical OR	`(5 > 3 || 4 > 2)`	Logical OR
`!`	Logical NOT	`!(5 > 3)`	false

4. Assignment Operators

Used to assign values to variables.

Operator	Meaning	Example	Equivalent To
`=`	Assignment	`x = 5`	—
`+=`	Add and assign	`x += 5`	`x = x + 5`
`-=`	Subtract and assign	`x -= 3`	`x = x - 3`
`*=`	Multiply and assign	`x *= 2`	`x = x * 2`
`/=`	Divide and assign	`x /= 2`	`x = x / 2`
`%=`	Modulus and assign	`x %= 2`	`x = x % 2`

5. Unary Operators

Works with a single operand.

Operator	Meaning	Example	Result
`+`	Unary plus (positive value)	`+a`	+a
`-`	Unary minus (negative value)	`-a`	-a
`++`	Increment	`a++` or `++a`	a = a + 1
`--`	Decrement	`a--` or `--a`	a = a - 1
`!`	Logical NOT	`!true`	false

6. Ternary Operator (Shortcut for if-else)

```
variable = (condition) ? value_if_true : value_if_false;
```

Example:

```
int age = 18;
String result = (age >= 18) ? "Adult" : "Minor";
System.out.println(result);  // Output: Adult
```

Simple Practice Questions (for Beginners)

1. **Take two numbers from the user and print their sum, difference, product, and division.**
2. **Write a program to check if a number is even or odd using `%` and `if` condition.**
3. **Take a user's age and print "Adult" or "Minor" using the ternary operator.**
4. **Use logical operators to check if a person can vote (age ≥ 18 and has voter ID).**

Java Math Library

The `Math` class in Java belongs to the `java.lang` package and provides many static methods for performing **mathematical operations** like square root, trigonometry, logarithms, rounding, etc.

You don't need to import this class separately as it's part of the default `java.lang` package.
All methods are **static**, so you can use them **directly with the class name**:
`Math.methodName()`.

Commonly Used Methods in `Math` Class

Method	Description	Example
`Math.abs(x)`	Returns absolute value of x	`Math.abs(-5) → 5`
`Math.max(a, b)`	Returns larger value	`Math.max(4, 7) → 7`
`Math.min(a, b)`	Returns smaller value	`Math.min(4, 7) → 4`
`Math.sqrt(x)`	Returns square root of x	`Math.sqrt(25) → 5.0`
`Math.pow(a, b)`	Returns a raised to the power of b	`Math.pow(2, 3) → 8.0`
`Math.cbrt(x)`	Returns cube root of x	`Math.cbrt(27) → 3.0`
`Math.round(x)`	Rounds to nearest integer	`Math.round(4.6) → 5`
`Math.ceil(x)`	Rounds **upward**	`Math.ceil(4.3) → 5.0`
`Math.floor(x)`	Rounds **downward**	`Math.floor(4.9) → 4.0`
`Math.random()`	Returns a random double between 0.0 and 1.0	`Math.random()`
`Math.log(x)`	Returns natural logarithm (base e)	`Math.log(1) → 0.0`
`Math.log10(x)`	Returns logarithm base 10	`Math.log10(100) → 2.0`
`Math.sin(x)`	Returns sine of angle in radians	`Math.sin(Math.PI/2) → 1.0`
`Math.cos(x)`	Returns cosine of angle	`Math.cos(0) → 1.0`
`Math.tan(x)`	Returns tangent of angle	`Math.tan(Math.PI/4) → 1.0`

Constants in Math Class

Constant	Value
`Math.PI`	3.141592653589793
`Math.E`	2.718281828459045

Example Java Program

```
public class MathDemo {
    public static void main(String[] args) {
        System.out.println("Absolute: " + Math.abs(-10));
        System.out.println("Max: " + Math.max(5, 9));
        System.out.println("Power: " + Math.pow(2, 3));
        System.out.println("Random number: " + Math.random());
        System.out.println("Sine of 90 degrees: " +
Math.sin(Math.toRadians(90)));
    }
}
```

Important Points

- All methods are **static**.
- Angles used in trigonometric functions must be in **radians**.
- `Math.random()` gives a value between **0.0 (inclusive)** and **1.0 (exclusive)**.

Loops in Java

Types of Loops:

1. `for` loop
2. `while` loop
3. `do-while` loop

1. `for` Loop

Used when: Number of iterations is known in advance.

Syntax:

```
for (initialization; condition; update) {
    // code block
}
```

Example:

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

2. `while` Loop

Used when: Number of iterations is **not known** in advance.

Syntax:

```
while (condition) {
    // code block
}
```

Example:

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

3. `do-while` Loop

Used when: Loop should run **at least once**.

Syntax:

```
do {
    // code block
} while (condition);
```

Example:

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i<= 5);
```

Loop Control Statements

Keyword	Description
`break`	Terminates the loop immediately
`continue`	Skips current iteration and continues the loop

Example – `break`:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.println(i);
}
```

Example – `continue`:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.println(i);
}
```

If else statements in Java

In Java, **`if-else` statements** are used to perform **decision-making operations**. It lets your program take different actions based on **conditions**.

Syntax of `if`, `if-else`, and `if-else if` in Java

1. Simple if statement

```
if (condition) {
    // Code to execute if condition is true
}
```

Example:

```
int age = 18;
if (age >= 18) {
    System.out.println("You are eligible to vote.");
}
```

2. if-else statement

Example:

```
int age = 16;
if (age >= 18) {
    System.out.println("You are eligible to vote.");
} else {
    System.out.println("You are not eligible to vote.");
}
```

3. if-else if-else ladder

Example:

```
int marks = 75;

if (marks >= 90) {
    System.out.println("Grade: A");
} else if (marks >= 75) {
    System.out.println("Grade: B");
} else if (marks >= 60) {
    System.out.println("Grade: C");
} else {
    System.out.println("Grade: D");
}
```

Notes:

- The **condition** must return a boolean value (`true` or `false`).
- You can nest `if-else` blocks inside each other.
- Java uses **curly braces `{}`** to group multiple statements, though for one-line statements, they are optional (not recommended for beginners).

`Switch case` Statement in Java

The `switch` statement is used to execute **one block of code among many options**. It is a better alternative to using many `if-else-if` statements when you're checking **a single variable** against multiple constant values.

Important Points:

- The `expression` should be **byte, short, int, char**, `String` (since Java 7), or enum type.
- `case` values must be **unique constants**.
- `break` is used to **exit** the switch block after a match is found. If not used, execution will "fall through" to the next case.
- `default` is **optional**, and it runs if no `case` matches.

Example 1: Using `int`

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Invalid day");
}
```

Output:

```
Wednesday
```

Example 2: Using `String` (Java 7+)

```
String fruit = "Apple";
switch (fruit) {
    case "Apple":
        System.out.println("Red fruit");
        break;
    case "Banana":
        System.out.println("Yellow fruit");
        break;
    default:
        System.out.println("Unknown fruit");
}
```

Example 3: Without `break` (fall-through behavior)

```
int num = 2;
switch (num) {
    case 1:
        System.out.println("One");
    case 2:
        System.out.println("Two");
    case 3:
        System.out.println("Three");
}
```

Output:

```
Two
Three
```

Because there's no `break`, it continues executing all the following cases.

When to Use:

Use `switch` when you have:

- One variable to check
- Multiple possible constant values to match

String Methods in Java

In Java, the `String` class provides many **useful methods** to manipulate and process text. Below is a list of commonly used methods with examples.

1. `length()`

Returns the number of characters in the string.

```
String str = "Hello";
System.out.println(str.length()); // Output: 5
```

2. charAt(int index)

Returns the character at the specified index.

```
String str = "Java";
System.out.println(str.charAt(2)); // Output: v
```

3. substring(int beginIndex)

Returns a substring from the given index to the end.

```
String str = "Programming";
System.out.println(str.substring(3)); // Output: gramming
```

4. substring(int beginIndex, int endIndex)

Returns a substring from `beginIndex` to `endIndex - 1`.

```
System.out.println(str.substring(0, 4)); // Output: Prog
```

5. equals(String anotherString)

Checks if two strings are **exactly equal** (case-sensitive).

```
String a = "hello";
String b = "hello";
```

```
System.out.println(a.equals(b)); // Output: true
```

6. equalsIgnoreCase(String anotherString)

Checks equality **ignoring case**.

```
String a = "Hello";
String b = "hello";
System.out.println(a.equalsIgnoreCase(b)); // Output: true
```

7. toUpperCase(), toLowerCase()

Converts the string to upper/lower case.

```
String str = "Java";
System.out.println(str.toUpperCase()); // Output: JAVA
System.out.println(str.toLowerCase()); // Output: java
```

8. contains(CharSequence seq)

Returns true if the sequence is present in the string.

```
String str = "Welcome";
System.out.println(str.contains("come")); // Output: true
```

9. replace(char oldChar, char newChar)

Replaces characters in the string.

```
String str = "banana";
System.out.println(str.replace('a', 'o')); // Output: bonono
```

10. trim()

Removes whitespace from both ends of the string.

```
String str = "  Hello World  ";
System.out.println(str.trim()); // Output: Hello World
```

11. startsWith() / endsWith()

Checks if string starts/ends with a given prefix/suffix.

```
String str = "hello.java";
System.out.println(str.startsWith("hello")); // true
System.out.println(str.endsWith(".java"));   // true
```

12. indexOf() / lastIndexOf()

Returns index of first/last occurrence of a character or substring.

```
String str = "programming";
```

```
System.out.println(str.indexOf('g'));      // Output: 3
System.out.println(str.lastIndexOf('g'));  // Output: 10
```

String Comparison in Java (== vs equals)

1. Using == Operator

- == checks **reference (memory address)**, not content.
- It returns `true` only if both references point to the **same object** in memory.
- Works fine with **primitives**, but not reliable for **Strings**.

Example:

```
String s1 = "Hello";
String s2 = "Hello";
System.out.println(s1 == s2);  // true (both in String Constant Pool)
```

Example:

```
String s3 = new String("Hello");
String s4 = new String("Hello");
System.out.println(s3 == s4);  // false (different objects in Heap)
```

2. Using `.equals()` Method

- `.equals()` checks **content (values)** of two strings.
- Returns `true` if both strings have the same sequence of characters.
- Best way to compare **string values** in Java.

Example:

```
String s1 = "Hello";
String s2 = new String("Hello");
System.out.println(s1.equals(s2));  // true (same content)
```

Nested Loops in Java

A **nested loop** means a loop **inside another loop**. It's useful for working with patterns, matrices, or multi-level iterations.

Syntax:

```
for (initialization; condition; update) {
    for (initialization; condition; update) {
        // inner loop body
    }
    // outer loop body
}
```

You can nest any kind of loop:

- `for` inside `for`
- `while` inside `for`

- `do-while` inside `while`, etc.

Example 1: Nested `for` loop (Print a rectangle of stars)

```
for (int i = 1; i <= 3; i++) {           // outer loop (rows)
    for (int j = 1; j <= 5; j++) {       // inner loop
(columns)
        System.out.print("* ");
    }
    System.out.println();               // move to the next
line
}
```

Output:

```
* * * * *
* * * * *
* * * * *
```

Associativity in Java

Definition

Associativity in Java defines **the order in which operators of the same precedence are evaluated**.
When two or more operators with the **same precedence** appear in an expression, **associativity** tells us whether Java will evaluate them from **left-to-right** or **right-to-left**.

OPERATOR	TYPE	ASSOCIAVITY
( ) [ ] . ->		left-to-right
++ -- + - ! ~ (*type*) * & sizeof	Unary Operator	right-to-left
* / %	Arithmetic Operator	left-to-right
+ -	Arithmetic Operator	left-to-right
<< >>	Shift Operator	left-to-right
< <= > >=	Relational Operator	left-to-right
== !=	Relational Operator	left-to-right
&	Bitwise AND Operator	left-to-right
^	Bitwise EX-OR Operator	left-to-right
|	Bitwise OR Operator	left-to-right
&&	Logical AND Operator	left-to-right
||	Logical OR Operator	left-to-right
? :	Ternary Conditional Operator	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Operator	right-to-left
,	Comma	left-to-right

Example:

```
int result = 100 / 5 * 2;
// First: 100 / 5 = 20
// Then: 20 * 2 = 40
// Output = 40
```

Resulting Data Type in Java (Type Promotion in Expressions)

Definition

In Java, when arithmetic operations are performed between **different data types**, the smaller type is **promoted** to a larger type before evaluation.

This rule is called **Type Promotion** or **Type Conversion in Expressions**.

Key Rules:

- **byte, short, and char → promoted to int** before any operation.
  - Example:
  - `short s1 = 5, s2 = 10;`
  - `int result = s1 + s2; // result is int, not short`
- **If one operand is long → result becomes long**.
  - Example:
  - `int a = 10;`
  - `long b = 20;`
  - `long result = a + b; // result is long`
- **If one operand is float → result becomes float**.

Common Examples:

- **int + short → int**
- `int a = 5;`
- `short b = 2;`
- `int result = a + b; // result is int`
- **byte + byte → int**
- `byte x = 10, y = 20;`
- `int result = x + y; // result is int`
- **char + int → int**
- `char c = 'A'; // Unicode 65`
- `int result = c + 1; // 65 + 1 = 66, result is int`
- **int + long → long**
- `int a = 10;`
- `long b = 100L;`
- `long result = a + b; // result is long`
- **long + float → float**
- `long l = 100L;`
- `float f = 3.5f;`
- `float result = l + f; // result is float`

Type Promotion Hierarchy:

```
byte → short → int → long → float → double
```

When different types are mixed in an operation, Java **promotes them to the largest type in the chain**.

Arrays in Java

1. Definition

- An array in Java is a **collection of elements of the same data type** stored in a contiguous memory location.
- It is used to store multiple values in a single variable, instead of declaring separate variables for each value.

2. Key Features

- Fixed size: The size of an array is defined at the time of creation and cannot be changed.

- Homogeneous elements: All elements must be of the same type.
- Index-based: Each element can be accessed by its index (starting from `0`).

3. Declaration and Initialization

There are two ways:

Declaration:

```
int[] arr;    // preferred way
int arr[];    // also valid
```

Memory allocation:

```
arr = new int[5];   // creates an array of size 5
```

Declaration + Initialization together:

```
int[] arr = new int[5];    // default values (0 for int, null for objects)
```

Initialization with values:

```
int[] arr = {10, 20, 30, 40, 50};
```

4. Accessing Elements

- Using index:

```
System.out.println(arr[0]);   // prints first element
```

- Changing value:

```
arr[2] = 100;   // updates 3rd element
```

5. Array Length

- `arr.length` gives the total size of the array.

```
System.out.println(arr.length);
```

6. Types of Arrays

1. **One-Dimensional Array**

   int[] arr = {1, 2, 3, 4, 5};

2. **Two-Dimensional Array (Matrix)**

   int[][] matrix = { {1,2}, {3,4}, {5,6} };

   Access element:

   ```
   System.out.println(matrix[1][0]);  // prints 3
   ```

3. **Multidimensional Array**

   Arrays inside arrays (3D or more).

```
int[][][] arr3D = new int[2][3][4];
```

7. Traversing Arrays

- Using **for loop**:

```
for(int i=0; i<arr.length; i++) {
    System.out.println(arr[i]);
}
```

- Using **for-each loop**:

```
for(int num : arr) {
    System.out.println(num);
}
```

8. Default Values in Arrays

- Numeric types → `0`
- char → `'\u0000'` (null character)
- boolean → `false`
- objects → `null`

9. Advantages

- Stores multiple values under one name.
- Easy to access using indexes.
- Better performance for fixed-size data.

10. Limitations

- Fixed size (cannot grow/shrink dynamically).
- Only stores elements of the same type.
- For dynamic data, **ArrayList** is preferred.

What is a 2D Array in Java

- A **2D array** is like a **table (rows and columns)**.
- It is an **array of arrays**.
  Example: `int[][] arr = new int[3][4];`
  → This creates a table with 3 rows and 4 columns.

Declaration of 2D Array

There are two ways:

1. **Declaration + Size Allocation**

```
int[][] arr = new int[3][4];
```

- 3 rows, 4 columns
- Default values = `0`

2. **Declaration + Initialization**

```
int[][] arr = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

- 3x3 matrix initialized directly.

Accessing Elements

```
arr[0][2];   // Row 0, Column 2 → 3
arr[2][1];   // Row 2, Column 1 → 8
```

Traversing a 2D Array

Using **nested for loops**:

```
for (int i = 0; i < arr.length; i++) {           // rows
    for (int j = 0; j < arr[i].length; j++) {    // columns
        System.out.print(arr[i][j] + " ");
    }
    System.out.println();
}
```

Input a 2D Array (Using Scanner)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[][] arr = new int[2][3]; // 2 rows, 3 columns

        // Input
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 3; j++) {
                arr[i][j] = sc.nextInt();
            }
        }

        // Output
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(arr[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Key Points

- `arr.length` → number of rows
- `arr[i].length` → number of columns in row `i`
- Default values in int array = `0`
- 2D arrays can also be **jagged** (rows having different column sizes).

Arrays Utility Class in Java

Java provides a built-in class **`java.util.Arrays`** that contains **static methods** to work with arrays.
Instead of writing code from scratch, we can use this class to perform **common operations** like sorting, searching, comparing, and filling arrays.

Common Methods of **`Arrays`** Class

1. `sort()`

- Sorts the array elements in ascending order.

```
import java.util.Arrays;

public class Demo {
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 1};
        Arrays.sort(arr);
        System.out.println(Arrays.toString(arr)); // [1, 2, 5, 8]
    }
}
```

2. `binarySearch()`

- Searches an element in a **sorted array**.
- Returns **index** if found, otherwise returns **negative value**.

```
int[] arr = {1, 2, 5, 8};
int index = Arrays.binarySearch(arr, 5);
System.out.println(index); // 2
```

3. `equals()`

- Compares two arrays (element by element).
- Returns true if both arrays are of same length and content.

```
int[] a1 = {1, 2, 3};
int[] a2 = {1, 2, 3};
System.out.println(Arrays.equals(a1, a2)); // true
```

4. `fill()`

- Fills the entire array with a given value.

```
int[] arr = new int[5];
Arrays.fill(arr, 10);
System.out.println(Arrays.toString(arr)); // [10, 10, 10, 10, 10]
```

5. `copyOf() and copyOfRange()`

- Copies elements from one array into another.

```
int[] arr = {1, 2, 3, 4, 5};
int[] copy = Arrays.copyOf(arr, 3); // [1, 2, 3]
```

```
int[] range = Arrays.copyOfRange(arr, 1, 4); // [2, 3, 4]
```

6. `toString()`

- Returns a string representation of the array.

```
int[] arr = {1, 2, 3};
System.out.println(Arrays.toString(arr)); // [1, 2, 3]
```

Methods in Java

- A **method** is a block of code that performs a specific task.
- It helps in **code reusability** and **better organization**.
- Java methods are similar to functions in C/C++.

Syntax of a Method

```
returnType methodName(parameters) {
    // method body
    return value;  // if returnType is not void
}
```

Example:

```
int add(int a, int b) {
    return a + b;
}
```

Types of Methods

- **Predefined Methods (Built-in)**
  - Already provided by Java libraries.
  - **Example:**

    `Math.sqrt(25)` → returns 5.0
    `System.out.println("Hello")`

- **User-defined Methods**
  - Created by programmer as per requirement.
  - **Example:**

```
void greet() {
   System.out.println("Welcome to Java!");
}
```

Method Signature

- Method name + parameter list = **Method Signature**
  Example: `add(int a, int b)`

Method Calling:

To execute a method, we **call** it.

```
class Main {
    static void greet() {
        System.out.println("Hello!");
    }

    public static void main(String[] args) {
```

```
        greet(); // calling method
    }
}
```

Return Type:

`void` → no value returned.

Any data type (`int`, `double`, `String`, etc.) → returns value.

```
int square(int num) {
    return num * num;
}
```

Parameters and Arguments:

- **Parameters**: variables defined inside method declaration.
- **Arguments**: actual values passed while calling.

```
int sum(int a, int b) {  // parameters
    return a + b;
}

sum(5, 10);  // arguments
```

Method Overloading:

Same method name but **different parameters** (number or type).

```
int add(int a, int b) {
    return a + b;
}
double add(double a, double b) {
    return a + b;
}
```

Static vs Non-static Methods:

Static Method: Can be called without creating object.

Example: `Math.max(5, 10)`

Non-static Method: Needs object to be called.

```
class Test {
    void display() {
        System.out.println("Non-static method");
    }
    public static void main(String[] args) {
        Test obj = new Test();
        obj.display(); // object required
    }
}
```

What is Varargs

`Varargs` (variable-length arguments) allow a method to accept **zero or multiple arguments** of the same type.

Declared using `...` (three dots).

```
void printNumbers(int... nums) {
    for (int n : nums) {
        System.out.print(n + " ");
    }
}
```

Call:

```
printNumbers(1, 2, 3, 4);  // accepts multiple arguments
```

Method Overloading with Varargs:

We can overload methods that use varargs by **changing parameter types** or **number of parameters**.
But, **confusion may occur** if normal parameter methods and varargs look similar.

Example 1: Different Parameter Types

```
class Test {
    void show(int... a) {
        System.out.println("int varargs method");
    }
    void show(String... s) {
        System.out.println("String varargs method");
    }

    public static void main(String[] args) {
        Test t = new Test();
        t.show(1, 2, 3);        // calls int version
        t.show("A", "B", "C");  // calls String version
    }
}
```

Example 2: Normal Parameter + Varargs

```
class Test {
    void display(int a, int b) {
        System.out.println("Normal method: " + (a + b));
    }
    void display(int... nums) {
        System.out.println("Varargs method, count = " + nums.length);
    }

    public static void main(String[] args) {
        Test t = new Test();
        t.display(5, 10);          // calls normal method (exact match)
        t.display(1, 2, 3, 4, 5); // calls varargs method
    }
}
```

Recursion in Java

- **Recursion** is a process in which a method calls **itself** directly or indirectly.
- Used for solving problems that can be broken into **smaller sub-problems of the same type**.

Syntax of Recursive Method:

```
returnType methodName(parameters) {
    // base condition
    if (condition) {
        return value;
    }
    // recursive call
    return methodName(modifiedParameters);
}
```

Key Terms:

1. **Base Case** → The condition that stops recursion.
   (Without base case, recursion will go infinite → `StackOverflowError`).
2. **Recursive Case** → The part where method calls itself.

Example 1: Factorial Using Recursion

```
class RecursionDemo {
    static int factorial(int n) {
        if (n == 0 || n == 1) {   // base case
            return 1;
        }
        return n * factorial(n - 1);  // recursive call
    }

    public static void main(String[] args) {
        System.out.println(factorial(5)); // 120
    }
}
```

Example 2: Fibonacci Using Recursion

```
class RecursionDemo {
    static int fib(int n) {
        if (n <= 1) {   // base case
            return n;
        }
        return fib(n - 1) + fib(n - 2);  // recursive case
    }

    public static void main(String[] args) {
        System.out.println(fib(6)); // 8
    }
}
```

Object-Oriented Programming (OOP) in Java

Object-Oiented Programming (OOP) is a programming style that organizes a program into **objects**.

- **Object** = real-world entity (example: Student, Car, Employee).
- Each object has:
  - **Attributes (data/properties)** → what it has
  - **Methods (functions/behavior)** → what it does

Principles of OOP

1. **Encapsulation**
   - Wrapping data and methods together inside a class.
   - Provides **data hiding** → outside world can't directly access sensitive data.
   - Access through getters & setters.
2. **Inheritance**
   - One class can acquire properties and behaviors of another class.
   - Promotes **code reusability** and represents **IS-A relationship** (Dog IS-A Animal).
3. **Polymorphism**
   - "Many forms".
   - A single method/object behaves differently in different situations.
   - Two types:

    - Compile-time (Method Overloading)
    - Runtime (Method Overriding)
4. **Abstraction**
    - Hiding implementation details and showing only important features.
    - Achieved using **abstract classes** or **interfaces**.
    - Example: ATM machine shows buttons but hides internal working.

Creating a New Class in Java

1. What is a Class

- A **class** is a blueprint for creating objects.
- It contains **fields (variables)** and **methods (functions)**.

2. How to Create a Class

```
// Defining a new class
class Student {
    // Fields (variables)
    private String name;
    private int age;

    // Methods (functions)
    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
```

`Student` is the class, and inside it we have **variables** (`name`, `age`) and a **method** (`displayInfo`).

Getters and Setters in Java

1. Why use Getters and Setters?

- To follow **Encapsulation** (hiding data using `private` keyword).
- Direct access to variables is **not allowed**.
- We use **getters** to read values, and **setters** to update values safely.

2. Example: Getters and Setters

```
class Student {
    // Private fields (cannot be accessed directly)
    private String name;
    private int age;

    // Getter for name
    public String getName() {
        return name;
    }

    // Setter for name
    public void setName(String name) {
        this.name = name;
    }
```

```
    // Getter for age
    public int getAge() {
        return age;
    }

    // Setter for age
    public void setAge(int age) {
        this.age = age;
    }
}
```

3. Using the Class with Getters & Setters

```
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();

        // Setting values using setters
        s1.setName("Neeraj");
        s1.setAge(23);

        // Getting values using getters
        System.out.println("Name: " + s1.getName());
        System.out.println("Age: " + s1.getAge());
    }
}
```

Access Modifiers in Java

1. **public** → Accessible **from anywhere** (same class, same package, different package, subclasses).
2. **protected** → Accessible **within same package** and also **in subclasses (even if in different package)**.
3. **default (no modifier / package-private)** → Accessible only **within same package**.
4. **private** → Accessible only **within the same class**.

What is a Constructor?

- A **constructor** is a **special method** in Java that is used to **initialize objects**.
- It is automatically called when an object is created.
- Constructor ka naam **class ke naam jaisa hi hota hai** aur iska **koi return type nahi hota (not even void)**.

Key Points

1. Constructor name = Class name.
2. No return type.
3. Called automatically when `new` keyword is used.
4. Can be **overloaded** (multiple constructors with different parameters).
5. If you don't define a constructor → Java provides a **default constructor**.

Types of Constructors in Java

- **Default Constructor**

No parameters.
Provided by Java if we don't create any constructor.
Initializes objects with **default values**.

- **Example (conceptual):**

int = 0, float = 0.0, boolean = false, String/Object = null.

- **Parameterized Constructor**

Takes arguments to assign values to attributes.
Allows initialization of objects with specific values.

Constructor Overloading:

Having more than one constructor in the same class with different parameter lists.
Helps in creating objects in multiple ways.

Example:

```
// Class
class Student {
    String name;
    int age;

    // 1. Default Constructor
    Student() {
        name = "Unknown";
        age = 0;
        System.out.println("Default constructor called");
    }

    // 2. Parameterized Constructor
    Student(String n, int a) {
        name = n;
        age = a;
        System.out.println("Parameterized constructor called");
    }

    // Method to display student details
    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        // Object using default constructor
        Student s1 = new Student();
        s1.display();

        // Object using parameterized constructor
        Student s2 = new Student("Neeraj", 23);
        s2.display();
    }
}
```

Inheritance in Java

1. What is Inheritance?

- Inheritance is a concept where **one class (child class)** acquires the **properties and methods** of another class (parent class).
- It allows **code reusability** and supports **hierarchical relationships** in OOP.

In simple words: **"Child class can reuse parent's features."**

Example:

```
// Parent class
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

// Child class
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();   // Inherited from Animal
        d.bark();  // Defined in Dog
    }
}
```

Output:

```
This animal eats food.
Dog barks.
```

Types of Inheritance in Java

1. **Single Inheritance** – One class inherits another.
   (A → B)
2. **Multilevel Inheritance** – A class inherits another, which itself is inherited by a third.
   (A → B → C)
3. **Hierarchical Inheritance** – Multiple classes inherit a single parent class.
   (A → B, A → C)

Multiple Inheritance using classes is **not allowed** in Java (to avoid ambiguity). But it can be achieved using **interfaces**.

Important Keywords

- `extends` → Used to inherit a class.
- `super` → Refers to the parent class (used to call parent methods/constructors).

Advantages

- **Code reusability** (no need to rewrite code).
- **Method overriding** support (Polymorphism).
- Creates a **hierarchy** of classes.

Constructor in Inheritance

- In inheritance, **parent class constructor** always runs before the **child class constructor**.
- Reason: Parent's part of object must be initialized first.
- We use `super()` to explicitly call parent's constructor (default or parameterized).

Case 1: Default Constructor

```
class Employee {
    String name;
    double salary;

    // Default constructor
    Employee() {
        name = "Unknown";
        salary = 0.0;
        System.out.println("Employee constructor called");
    }
}

class Manager extends Employee {
    Manager() {
        System.out.println("Manager constructor called");
    }
}

public class Main {
    public static void main(String[] args) {
        Manager m = new Manager(); // Object created
        System.out.println("Name: " + m.name + ", Salary: " + m.salary);
    }
}
```

Output:

```
Employee constructor called
Manager constructor called
Name: Unknown, Salary: 0.0
```

Case 2: Parameterized Constructor with super()

```
class Employee {
    String name;
    double salary;

    // Parameterized constructor
    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
        System.out.println("Employee constructor called for " + name);
    }
}

class Manager extends Employee {
    String department;
```

```
    // Child constructor
    Manager(String name, double salary, String department) {
        super(name, salary); // Call parent constructor
        this.department = department;
        System.out.println("Manager constructor called for " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Manager m = new Manager("Neeraj", 50000, "IT");
        System.out.println("Name: " + m.name + ", Salary: " + m.salary + ",
Department: " + m.department);
    }
}
```

Output:

```
Employee constructor called for Neeraj
Manager constructor called for Neeraj
Name: Neeraj, Salary: 50000.0, Department: IT
```

1. `this` Keyword

- Refers to the **current object** of the class.
- Used to differentiate between **class variables** and **method/local variables** when they have the same name.

Uses of **`this`**

1. **Refers to current object**
2. **Differentiate variables** (when local variable & instance variable have same name)
3. **Call current class methods**
4. **Call current class constructor** (using `this()`)

Example with **`name`** and **`salary`**

```
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;       // "this" refers to instance variable
        this.salary = salary;
    }

    void display() {
        System.out.println("Name: " + this.name + ", Salary: " +
this.salary);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e1 = new Employee("Neeraj", 50000);
        e1.display();
    }
}
```

Output:

```
Name: Neeraj, Salary: 50000.0
```

2. super Keyword

- Refers to the **immediate parent class object**.
- Used in **inheritance** to access parent's variables, methods, and constructors.

Uses of super

1. **Access parent class variables** (if same name exists in child class).
2. **Call parent class methods** (if overridden in child class).
3. **Call parent class constructor** (first line of child constructor).

Example with name and salary

```
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
        System.out.println("Employee constructor called for " + name);
    }

    void display() {
        System.out.println("Employee: " + name + ", Salary: " + salary);
    }
}

class Manager extends Employee {
    String department;

    Manager(String name, double salary, String department) {
        super(name, salary); // calls parent constructor
        this.department = department;
        System.out.println("Manager constructor called for " + name);
    }

    @Override
    void display() {
        super.display(); // calls parent method
        System.out.println("Department: " + department);
    }
}

public class Main {
    public static void main(String[] args) {
        Manager m = new Manager("Rahul", 70000, "IT");
        m.display();
    }
}
```

Output:

```
Employee constructor called for Rahul
Manager constructor called for Rahul
Employee: Rahul, Salary: 70000.0
Department: IT
```

Method Overriding in Java

- **Method Overriding** happens when a **child class** provides its own implementation of a method that is already defined in the **parent class**.
- The **method signature** (name + parameters) must be **same**.
- Return type should be the same (or covariant in case of objects).

It is used to achieve **Runtime Polymorphism**.

Rules for Method Overriding

1. Method name and parameters must be the **same**.
2. Return type must be the **same** (or covariant).
3. Access modifier of child's method **cannot be more restrictive** than parent's.
    - Example: if parent method is `public`, child cannot make it `private`.
4. Only **inherited methods** can be overridden.
5. **Constructors cannot be overridden**.
6. Use `@Override` annotation for clarity.

Example

```
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void displayInfo() {
        System.out.println("Employee: " + name + ", Salary: " + salary);
    }
}

class Manager extends Employee {
    String department;

    Manager(String name, double salary, String department) {
        super(name, salary);
        this.department = department;
    }

    // Overriding the displayInfo method
    @Override
    void displayInfo() {
        System.out.println("Manager: " + name + ", Salary: " + salary + ",
Department: " + department);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Employee("Neeraj", 50000);
        Manager m = new Manager("Rahul", 70000, "IT");

        e.displayInfo();  // Parent class method
        m.displayInfo();  // Overridden method in Manager class
    }
}
```

Output:

```
Employee: Neeraj, Salary: 50000.0
Manager: Rahul, Salary: 70000.0, Department: IT
```

Key Point: super in Overriding

Child can still call **parent class method** using `super`:

```
@Override
void displayInfo() {
    super.displayInfo();  // calls parent method
    System.out.println("Department: " + department);
}
```

Abstract Class and Abstract Methods in Java

1. Abstract Class

- An **abstract class** in Java is declared using the **`abstract` keyword**.
- You **cannot create an object** of an abstract class directly.
- It works like a **blueprint** and can contain both:
    - **Abstract methods** (without body)
    - **Normal methods** (with body)

2. Abstract Method

- An **abstract method** only provides **declaration**, not implementation.
- The **implementation must be provided by a subclass**.
- Abstract methods **cannot have a body**.
- If a class contains an abstract method, that class must also be declared as **abstract**.

3. Key Points

- An abstract class **can have a constructor**.
- It can contain **both abstract and non-abstract methods**.
- If a subclass inherits an abstract class, it **must implement all abstract methods** (unless the subclass is also abstract).

Example:

```
// Abstract class
abstract class Employee {
    String name;
    double salary;

    // Constructor
    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
```

```
    // Abstract method
    abstract void work();

    // Normal method
    void displayDetails() {
        System.out.println("Name: " + name + ", Salary: " + salary);
    }
}

// Subclass
class Manager extends Employee {
    Manager(String name, double salary) {
        super(name, salary);
    }

    // Implementing abstract method
    void work() {
        System.out.println(name + " is managing the team.");
    }
}

// Another Subclass
class Developer extends Employee {
    Developer(String name, double salary) {
        super(name, salary);
    }

    void work() {
        System.out.println(name + " is writing code.");
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Employee e1 = new Manager("Rahul", 50000);
        Employee e2 = new Developer("Neha", 60000);

        e1.displayDetails();
        e1.work();

        e2.displayDetails();
        e2.work();
    }
}
```

Output

```
Name: Rahul, Salary: 50000.0
Rahul is managing the team.
Name: Neha, Salary: 60000.0
Neha is writing code.
```

Interface in Java

1. What is an Interface?

- An **interface** in Java is a collection of **abstract methods** (methods without body).
- It is used to **achieve abstraction** and **multiple inheritance** in Java.
- By default, all variables inside an interface are:

    - `public static final` (constants)
- All methods are:
    - `public abstract` (implicitly, even if you don't write them).

Syntax:

```
interface InterfaceName {
    // Abstract method
    void method1();

    // Variables (constants)
    int VALUE = 100;
}
```

2. Implementing an Interface

- A class **uses the `implements` keyword** to use an interface.
- The class must provide **implementation for all methods** in the interface.

Example:

```
interface Animal {
    void sound();  // abstract method
    void sleep();
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
    public void sleep() {
        System.out.println("Dog sleeps at night");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
        d.sleep();
    }
}
```

Output:

```
Dog barks
Dog sleeps at night
```

3. Multiple Inheritance with Interfaces

- A class can implement **multiple interfaces** (unlike classes, where multiple inheritance is not allowed).

Example:

```
interface Printable {
```

```
    void print();
}
interface Showable {
    void show();
}

class Demo implements Printable, Showable {
    public void print() {
        System.out.println("Printing...");
    }
    public void show() {
        System.out.println("Showing...");
    }
}

public class Main {
    public static void main(String[] args) {
        Demo obj = new Demo();
        obj.print();
        obj.show();
    }
}
```

Output:

```
Printing...
Showing...
```

4. Default and Static Methods in Interface (Java 8+)

- **Default methods**: Interfaces can have methods with a body using the `default` keyword.
- **Static methods**: Interfaces can also contain static methods.

Example:

```
interface Vehicle {
    void drive();

    default void fuel() {
        System.out.println("This vehicle uses fuel");
        price();
    }

    static void service() {
        System.out.println("Vehicle needs servicing");
    }
    private void price() {
        System.out.println("Vehicle price is:4000");
    }

}

class Car implements Vehicle {
    public void drive() {
        System.out.println("Car is driving");
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Car c = new Car();
        c.drive();
        c.fuel();       // default method
        Vehicle.service(); // static method
    }
}
```

Output:

```
Car is driving
This vehicle uses fuel
Vehicle needs servicing
```

5. Key Differences: Abstract Class vs Interface

Feature	Abstract Class	Interface
Keyword	`abstract`	`interface`
Methods	Abstract + Normal	Only Abstract (till Java 7), Abstract + Default + Static (Java 8+)
Variables	Any type	Always `public static final`
Multiple Inheritance	Not supported	Supported
Constructor	Allowed	Not allowed

Java as a Compiled Language vs Interpreted Language

Java is considered **both compiled and interpreted** because it uses a **two-step execution model**:

Compiled Language (Java Part)

- Java source code (`.java`) is first **compiled by `javac` (Java compiler)**.
- Compiler converts the code into **bytecode (`.class` files)**.
- Bytecode is **platform-independent** (can run on any OS with JVM).

Example:

```
javac MyProgram.java   // compilation step → MyProgram.class (bytecode)
```

Interpreted Language (Java Part)

- The **Java Virtual Machine (JVM)** reads and **interprets bytecode** line by line.
- In modern JVMs, the **JIT (Just-In-Time) compiler** converts bytecode into **native machine code** for faster execution.

Example:

```
java MyProgram   // JVM interprets and runs the bytecode
```

Summary:

- **Compiled** → `.java` → `.class` (bytecode)
- **Interpreted** → JVM executes bytecode line by line (or compiles it just-in-time).
- That's why Java is often called a **"compiled + interpreted language"**.

Packages in Java

What is a Package?

- A **package** in Java is like a **folder** that groups related classes, interfaces, and sub-packages together.
- Used to **organize code**, **avoid name conflicts**, and **provide access control**.

Think of it like folders on your computer:

```
com.techvision.utils
```

Here,

- `com` = top-level package
- `techvision` = sub-package
- `utils` = sub-package containing classes

Types of Packages

1. **Built-in Packages** (provided by Java)
    - Example:
        - `java.util` → Scanner, ArrayList
        - `java.io` → File, BufferedReader
2. **User-defined Packages** (created by programmers)
    - You can create your own packages for organizing project files.

Creating and Using a Package

Step 1 – Create a Package

```
// File: MyClass.java
package mypackage;   // declare package

public class MyClass {
    public void display() {
        System.out.println("Hello from MyClass in mypackage");
    }
}
```

Step 2 – Compile with package

```
javac -d . MyClass.java
// javac -d . *.java                // for all file
```

(`-d .` creates folder structure for package)

Step 3 – Use the Package

```
// File: Main.java
import mypackage.MyClass;   // import the package

public class Main {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.display();
    }
}
```

Benefits of Packages

- **Code Reusability** – Organized libraries.
- **Avoids Naming Conflicts** – Two classes with the same name can exist in different packages.
- **Access Protection** – `public`, `protected`, `default` access levels depend on package use.
- **Modularity** – Large projects are easier to manage.

Access Modifiers in Java

1. What are Access Modifiers?

- **Access Modifiers** in Java are **keywords** that define the **scope (visibility)** of classes, methods, variables, and constructors.
- They control **who can access what** in a program.

2. Types of Access Modifiers

Java provides **four main access modifiers**:

Modifier	Class	Package	Subclass	Other Packages
private	Yes	No	No	No
default (no keyword)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

3. Explanation with Examples

(i) private

- Members are **accessible only inside the same class**.
- Cannot be accessed outside the class, not even by subclasses.

Example:

```
class A {
    private int data = 40;
    private void show() {
        System.out.println("Private method");
    }
}

public class Main {
    public static void main(String[] args) {
        A obj = new A();
        // System.out.println(obj.data); // Error
        // obj.show(); // Error
    }
}
```

(ii) default (no modifier)

- When no modifier is specified, it is **package-private**.
- Accessible only **within the same package**.

Example:

```
class A {
    int data = 50; // default
    void show() {
        System.out.println("Default method");
    }
}

class B {
    public static void main(String[] args) {
        A obj = new A();
        System.out.println(obj.data); // Allowed (same package)
        obj.show();
    }
}
```

(iii) protected

- Accessible within the **same package** and also in **subclasses of other packages**.

Example:

```
package pack1;
public class A {
    protected void display() {
        System.out.println("Protected method in A");
    }
}

package pack2;
import pack1.A;

class B extends A {
    public static void main(String[] args) {
        B obj = new B();
        obj.display(); // Allowed (subclass in different package)
    }
}
```

(iv) public

- Accessible **from anywhere** in the program.

Example:

```
class A {
    public void show() {
        System.out.println("Public method");
    }
}

public class Main {
    public static void main(String[] args) {
        A obj = new A();
        obj.show(); // Accessible everywhere
    }
}
```

4. Quick Summary Table

Modifier	Scope
private	Only within the same class
default	Within the same package only
protected	Within same package + subclasses in other packages
public	Accessible everywhere

Multithreading in Java

- **Multithreading** in Java is the ability to run **multiple parts of a program (threads)** at the same time.
- A **thread** is the smallest independent unit of execution in a program.
- Java supports multithreading by providing the **`Thread` class** and the **`Runnable` interface**.

Advantages of Multithreading

1. **Efficient CPU usage** – prevents CPU from staying idle.
2. **Faster execution** – multiple tasks run in parallel.
3. **Better responsiveness** – useful in GUI or server applications.
4. **Easy asynchronous execution** – tasks can run in background.

Creating Threads in Java

1. By Extending `Thread` class

- We can create a new class that **extends `Thread`** and overrides its `run()` method.
- Then create an object and call `start()` method.

```
class MyThread1 extends Thread {
```

```
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("MyThread1 running: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread1 t1 = new MyThread1(); // create object of thread
        t1.start();  // starts the thread, internally calls run()

    }
}
```

Here `run()` contains the task of the thread. `start()` is always used to begin execution, not `run()` directly.

2. Multiple Threads Example

- We can create multiple threads that run **simultaneously**.
- Output order is **not guaranteed** because threads run in parallel.

```
class Task1 extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Task1 running: " + i);
        }
    }
}

class Task2 extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Task2 running: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Task1 t1 = new Task1();
        Task2 t2 = new Task2();

        t1.start(); // start first thread
        t2.start(); // start second thread
    }
}
```

Both threads run together, so the output may look **mixed** like:

```
Task1 running: 1
Task2 running: 1
Task1 running: 2
Task2 running: 2
...
```

3. By Implementing `Runnable` interface

- Another way is to create a class that **implements** `Runnable` and defines the `run()` method.

- Then create a `Thread` object and pass the `Runnable` object to it.

```
class Task1 implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Task1 running: " + i);
        }
    }
}

class Task2 implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Task2 running: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // Create runnable objects
        Task1 task1 = new Task1();
        Task2 task2 = new Task2();

        // Wrap runnable objects inside Thread objects
        Thread t1 = new Thread(task1);
        Thread t2 = new Thread(task2);

        // Start both threads
        t1.start();
        t2.start();
    }
}
```

Output (order may vary)

```
Task1 running: 1
Task2 running: 1
Task1 running: 2
Task2 running: 2
...
```

This method is often preferred because Java supports **multiple inheritance through interfaces**, not classes.

Commonly Used Thread Methods

- `start()` → starts the execution of a thread.
- `run()` → contains the actual code for the thread.
- `sleep(ms)` → pauses the thread for given milliseconds.
- `join()` → waits for another thread to complete.
- `getName()` / `setName()` → gets/sets the name of a thread.
- `getPriority()` / `setPriority()` → manages priority (1–10).
- `isAlive()` → checks if the thread is still running.

Types of Errors in Java

1. Compile-time Errors

- These errors occur during compilation, i.e., before the program runs.
- The compiler checks for **syntax and structural mistakes** in the code.
- Common causes:
    - Missing semicolon `;`
    - Misspelled keywords
    - Type mismatch (assigning `int` to `String`)
    - Missing return statement in a non-void method

Example:

```
public class Main {
    public static void main(String[] args) {
        int x = "hello"; // Compile-time error: incompatible types
    }
}
```

2. Run-time Errors

- These occur **while the program is executing**.
- Even if the code compiles successfully, it can still fail at runtime.
- Common causes:
    - Dividing a number by zero
    - Accessing an array index out of bounds
    - Null pointer exception
    - File not found

Example:

```
public class Main {
    public static void main(String[] args) {
        int a = 5 / 0; // Runtime Error: ArithmeticException
    }
}
```

3. Logical Errors

- The program compiles and runs, but **the output is incorrect**.
- These errors happen due to **wrong logic** applied by the programmer.
- The compiler cannot detect these.

Example:

```
public class Main {
    public static void main(String[] args) {
        int a = 5, b = 10;
        int avg = (a + b) / 2;
        System.out.println("Average: " + avg); // Correct output: 7
        // If programmer mistakenly writes:
        int wrongAvg = (a - b) / 2;
        System.out.println("Average: " + wrongAvg); // Logical error: -2
    }
}
```

4. Linker Errors (Rare in Java)

- Usually happen if **external libraries, classes, or methods are missing** during linking.

- Example: Calling a method from a library that is not included in the classpath.

5. Exceptions vs Errors

- In Java, **both are part of Throwable class**, but they differ:

  Exception: Conditions that can be handled (e.g., file not found, divide by zero).

  Error: Serious problems that cannot be handled by the program (e.g., `OutOfMemoryError`, `StackOverflowError`).

Example:

```
public class Main {
    public static void main(String[] args) {
        // Example of Error (cannot handle)
        // throw new OutOfMemoryError("Memory exhausted");

        // Example of Exception (can handle using try-catch)
        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero!");
        }
    }
}
```

Try-Catch Block in Java

- `try-catch` is used in **exception handling** to prevent program termination when an error occurs.
- The **`try` block** contains code that might throw an exception.
- The **`catch` block** handles the exception.

Example 1: Simple `try-catch`

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10 / 0; // risky code
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero!");
        }
        System.out.println("Program continues...");
    }
}
```

Output:

```
Cannot divide by zero!
Program continues...
```

Example 2: Multiple Catch Blocks

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;  // ArithmeticException
```

```
            int[] arr = new int[5];
            arr[10] = 20;   // ArrayIndexOutOfBoundsException
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero!");
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index out of range!");
        }
        catch (Exception e) { // generic handler
            System.out.println("Some other error occurred: " + e);
        }
    }
}
```

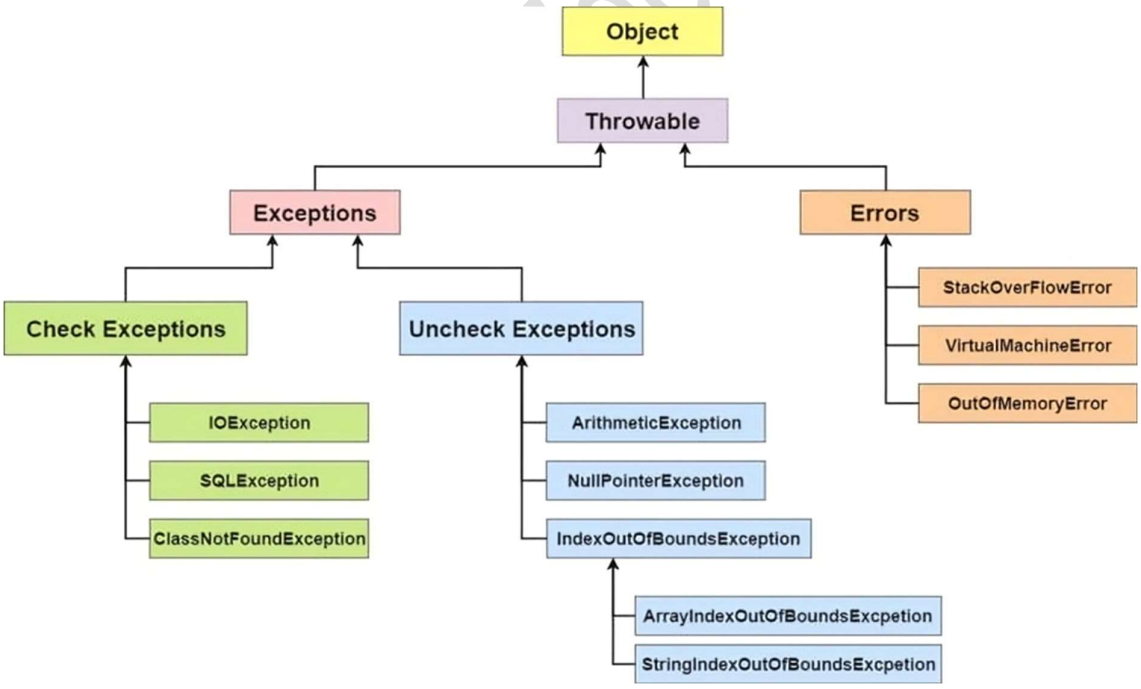
Output:

```
Cannot divide by zero!
```

Exception Class in Java

An *Exception* is an unwanted or unexpected event that occurs during the execution of a program, disrupting the normal flow of instructions. In Java, all exceptions are objects and are derived from the **`Throwable`** class.

Exception Hierarchy:



Key Points:

- **Throwable** → Parent class of all errors and exceptions.
- **Exception** → Represents conditions a program might want to catch.
- **RuntimeException** → Subclass of Exception (unchecked exceptions).
- **Error** → Serious problems that a program should not try to handle (like JVM crashes).

`throw` Keyword

The `throw` keyword is used to **explicitly throw** an exception from a method or block of code.

Syntax:

```
throw new ExceptionType("Error Message");
```

Points to Remember:
Only **one exception** can be thrown at a time using `throw`.
The object thrown must be of type **Throwable** (Exception or subclass).
Commonly used to throw custom or predefined exceptions.

Example:

```
class Test {
    public static void main(String[] args) {
        int age = 15;
        if (age < 18) {
            throw new ArithmeticException("Not eligible for voting");
        }
        System.out.println("Eligible for voting");
    }
}
```

`throws Keyword`

The `throw` keyword is used to **explicitly throw** an exception from a method or block of code.

Syntax:

```
throw new ExceptionType("Error Message");
```

Points to Remember:

Only **one exception** can be thrown at a time using `throw`.
The object thrown must be of type **Throwable** (Exception or subclass).
Commonly used to throw custom or predefined exceptions.

```
class Division {
    // Method declares it might throw an exception
    int divide(int a, int b) throws ArithmeticException {
        if (b == 0) {
            throw new ArithmeticException("Division by zero is not
allowed");
        }
        return a / b;
    }
}

public class ThrowsExample {
    public static void main(String[] args) {
        Division obj = new Division();
        try {
            int result = obj.divide(10, 0);
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```

Output:

```
Exception caught: Division by zero is not allowed
```

Default Behavior of getMessage() and toString()

- `getMessage()` → returns the string passed in `super(message)`.
- `toString()` → returns `"ClassName: message"`.

Example (Default):

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String msg) {
        super(msg); // Parent Exception stores the message
    }
}

public class Test {
    public static void main(String[] args) {
        try {
            throw new InvalidAgeException("Age must be 18+");
        } catch (InvalidAgeException e) {
            System.out.println("getMessage(): " + e.getMessage());
            System.out.println("toString(): " + e);
        }
    }
}
```

Output:

```
getMessage(): Age must be 18+
toString(): InvalidAgeException: Age must be 18+
```

Overriding getMessage() and toString()

If you want **custom formatting** for error messages, you can override these methods.

Example (Overridden):

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String msg) {
        super(msg);
    }

    @Override
    public String getMessage() {
        return "Custom Message → " + super.getMessage();
    }

    @Override
    public String toString() {
        return "InvalidAgeException occurred → " + super.getMessage();
    }
}

public class Test {
    public static void main(String[] args) {
        try {
            throw new InvalidAgeException("Age is below 18");
        } catch (InvalidAgeException e) {
            System.out.println("getMessage(): " + e.getMessage());
            System.out.println("toString(): " + e);
        }
    }
```

```
}
```

Output:

```
getMessage(): Custom Message → Age is below 18
toString(): InvalidAgeException occurred → Age is below 18
```

`finally` block in Java

- The `finally` block is used in exception handling.
- It contains **cleanup code** that **always executes** after `try` and `catch`, whether an exception occurs or not.

Key Points

1. Runs after `try` / `catch` — no matter what.
2. Executes even if there is a `return` in `try` or `catch`.
3. **Does not run** if JVM exits (`System.exit()`) or crashes.
4. Best used for **closing files, DB connections, releasing resources**.
5. Avoid writing `return` inside `finally` (it overrides the original return/exception).

Example

```
public class FinallyDemo {
    public static void main(String[] args) {
        try {
            System.out.println("In try");
            int x = 10 / 0;  // exception
        } catch (ArithmeticException e) {
            System.out.println("In catch");
        } finally {
            System.out.println("In finally");
        }
    }
}
```

Output:

```
In try
In catch
In finally
```

Java Collection Framework

1. Introduction

- The **Java Collection Framework (JCF)** provides a set of classes and interfaces to store and manipulate groups of objects.
- It is present in **`java.util` package**.
- It provides **ready-to-use data structures** like lists, sets, queues, maps, etc.
- Helps in writing **reusable, efficient, and maintainable code**.

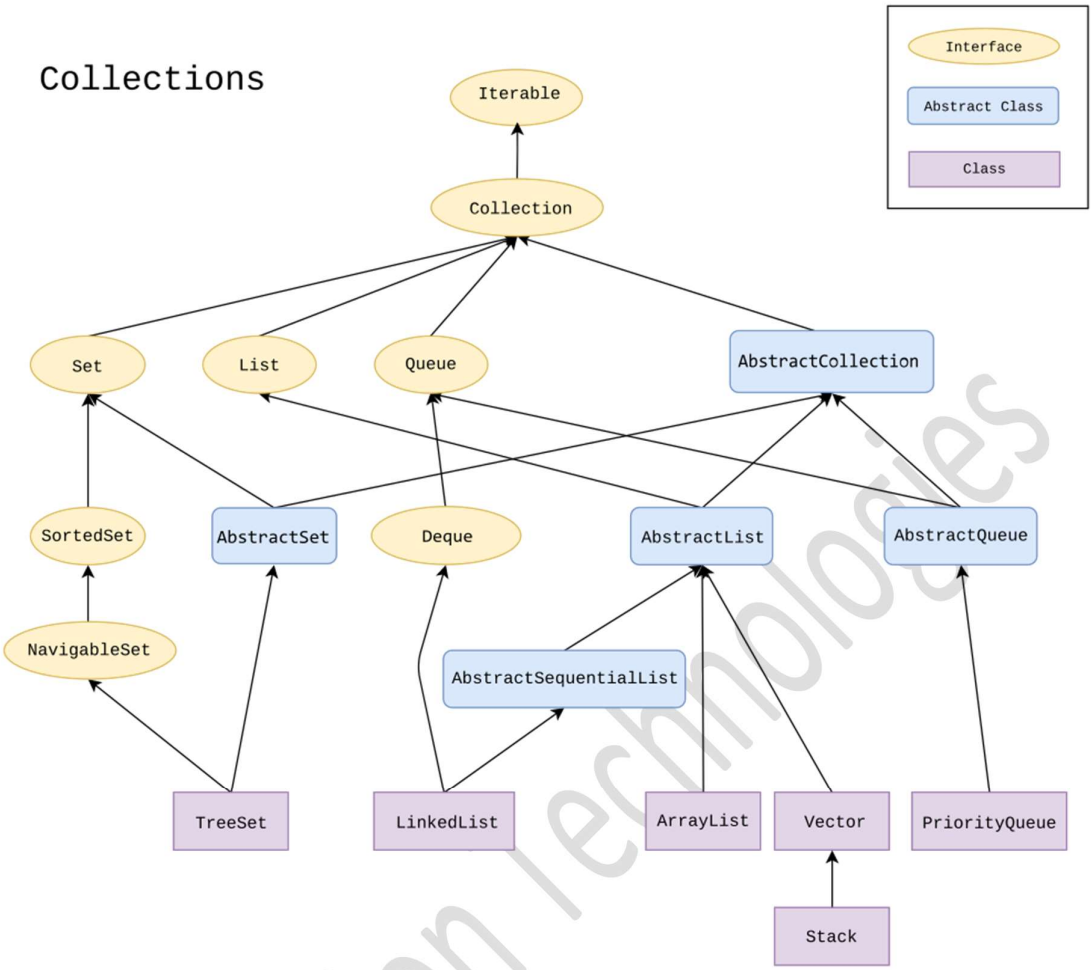
2. Key Features

- **Interfaces**: Provide abstract data types (List, Set, Queue, Map).
- **Implementations**: Concrete classes (ArrayList, HashSet, PriorityQueue, HashMap).
- **Algorithms**: Utility methods (sorting, searching) in `Collections` class.
- **Polymorphism**: You can use interface reference and change the implementation easily.

3. Core Interfaces of Collection Framework

- **Collection (root interface)**
  - Base interface for all collections (except Map).
  - Methods: `add()`, `remove()`, `size()`, `iterator()`.
- **List (extends Collection)**
  - Ordered, allows duplicate elements.
  - Implementations: `ArrayList`, `LinkedList`, `Vector`, `Stack`.
- **Set (extends Collection)**
  - No duplicates allowed.
  - Implementations: `HashSet`, `LinkedHashSet`, `TreeSet`.
- **Queue (extends Collection)**
  - Follows FIFO order (mostly).
  - Implementations: `PriorityQueue`, `ArrayDeque`, `LinkedList`.
- **Map (separate interface, not part of Collection)**
  - Stores key-value pairs, keys are unique.
  - Implementations: `HashMap`, `LinkedHashMap`, `TreeMap`, `Hashtable`.

4. Collection Framework Hierarchy



5. Important Classes

- **ArrayList**: Dynamic array, fast random access.
- **LinkedList**: Doubly linked list, efficient insert/delete.
- **Vector**: Legacy class, synchronized.
- **Stack**: LIFO stack (extends Vector).
- **HashSet**: No duplicates, unordered.
- **LinkedHashSet**: Maintains insertion order.
- **TreeSet**: Sorted set (ascending order by default).
- **HashMap**: Key-value pairs, no order.
- **LinkedHashMap**: Maintains insertion order.
- **TreeMap**: Sorted key-value pairs.
- **PriorityQueue**: Elements processed based on priority.
- **ArrayDeque**: Double-ended queue.

6. Utility Class

`Collections` class: Provides static methods for algorithms.

`sort(list)` – Sorts the list.

`reverse(list)` – Reverses the list.

`shuffle(list)` – Randomly shuffles elements.

`max(list),min(list)` – Finds maximum/minimum element.

ArrayList in Java

- **ArrayList** is a **resizable (dynamic) array** implementation of the `List` interface.
- Belongs to **`java.util` package**.
- Unlike normal arrays, ArrayList can **grow or shrink dynamically**.
- Allows **duplicate elements** and maintains **insertion order**.
- Provides **random access** (fast element retrieval by index).

Declaration:

```
ArrayList<Type> list = new ArrayList<>();
```

2. Constructors

- `ArrayList()` → Creates an empty ArrayList.
- `ArrayList(int capacity)` → Creates ArrayList with specified initial capacity.
- `ArrayList(Collection c)` → Creates ArrayList containing elements of another collection.

3. Commonly Used Methods

Adding Elements

- `add(E e)` → Appends element at the end.
- `add(int index, E e)` → Inserts element at given index.

```
list.add("Apple");
list.add(1, "Mango");
```

Accessing Elements

- `get(int index)` → Returns element at given index.

```
String fruit = list.get(0);
```

Modifying Elements

- `set(int index, E e)` → Replaces element at index with new value.

```
list.set(1, "Banana");
```

Removing Elements

- `remove(int index)` → Removes element at index.
- `remove(Object o)` → Removes first occurrence of given element.
- `clear()` → Removes all elements.

```
list.remove(0);
list.remove("Apple");
list.clear();
```

Searching Elements

- `contains(Object o)` → Returns `true` if element exists.
- `indexOf(Object o)` → Returns index of first occurrence (or -1 if not found).
- `lastIndexOf(Object o)` → Returns index of last occurrence.

```
boolean check = list.contains("Mango");
int pos = list.indexOf("Banana");
```

Size & Capacity

- `size()` → Returns number of elements.
- `isEmpty()` → Returns `true` if empty.
- `ensureCapacity(int minCapacity)` → Increases capacity if needed.
- `trimToSize()` → Trims capacity to current size.

```
int n = list.size();
boolean empty = list.isEmpty();
```

Iteration

- `iterator()` → Returns iterator.
- `listIterator()` → Returns list iterator (can traverse both directions).
- `forEach()` → Traverses using lambda.

```
for(String fruit : list) {
    System.out.println(fruit);
}
```

Conversion

- `toArray()` → Converts to array.

```
Object[] arr = list.toArray();
```

Example Code:

```
import java.util.*;

public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();

        // Adding elements
        fruits.add("Apple");
        fruits.add("Mango");
        fruits.add("Orange");

        // Access
        System.out.println(fruits.get(1)); // Mango

        // Modify
        fruits.set(1, "Banana");

        // Remove
        fruits.remove("Apple");

        // Search
        System.out.println(fruits.contains("Orange")); // true
```

```
        // Iteration
        for (String f : fruits) {
            System.out.println(f);
        }
    }
}
```

Summary:

- **ArrayList** = resizable array, maintains order, allows duplicates.
- Key methods = `add()`, `get()`, `set()`, `remove()`, `contains()`, `size()`, `iterator()`.
- Good for **random access**, but insertion/deletion in middle is slower than `LinkedList`.

LinkedList in Java

- **LinkedList** is a **doubly linked list** implementation of `List` and `Deque` interfaces.
- Belongs to **`java.util` package**.
- Maintains **insertion order**.
- Allows **duplicate elements**.
- Each element (called **Node**) contains:
    - **data**
    - **address of previous node**
    - **address of next node**

Unlike `ArrayList`, insertion and deletion are **faster**, but random access is **slower**.

Declaration:

```
LinkedList<Type> list = new LinkedList<>();
```

2. Constructors

`LinkedList()` → Creates an empty list.
`LinkedList(Collection c)` → Creates list with elements of another collection.

Commonly Used Methods

Adding Elements

- `add(E e)` → Adds element at the end.
- `add(int index, E e)` → Inserts element at index.
- `addFirst(E e)` → Inserts element at beginning.
- `addLast(E e)` → Inserts element at end.

```
list.add("A");
list.addFirst("Start");
list.addLast("End");
```

Accessing Elements

- `get(int index)` → Returns element at index.
- `getFirst()` → Returns first element.
- `getLast()` → Returns last element.

```
String s = list.get(1);
String first = list.getFirst();
```

Modifying Elements

- `set(int index, E e)` → Replaces element at index.

```
list.set(1, "NewValue");
```

Removing Elements

- `remove(int index)` → Removes element at index.
- `remove(Object o)` → Removes first occurrence.
- `removeFirst()` → Removes first element.
- `removeLast()` → Removes last element.
- `clear()` → Removes all elements.

```
list.remove("A");
list.removeFirst();
list.clear();
```

Queue/Deque Operations (because LinkedList implements `Deque`)

- `offer(E e)` → Adds element at end.
- `offerFirst(E e)` → Adds at beginning.
- `offerLast(E e)` → Adds at end.
- `poll()` → Retrieves and removes head.
- `pollFirst()` / `pollLast()` → Removes first/last element.
- `peek()` → Returns head without removing.
- `peekFirst()` / `peekLast()` → Returns first/last element without removing.

Size & Check

- `size()` → Returns number of elements.
- `isEmpty()` → Checks if empty.

Iteration

- `iterator()` → Forward iteration.
- `descendingIterator()` → Reverse iteration.
- Enhanced for loop (`for-each`).

Example Code

```
import java.util.*;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> names = new LinkedList<>();

        // Adding elements
        names.add("Neeraj");
        names.add("Rahul");
        names.addFirst("Amit");
        names.addLast("Suman");
```

```
        // Access
        System.out.println("First: " + names.getFirst());
        System.out.println("Last: " + names.getLast());

        // Modify
        names.set(1, "Updated");

        // Remove
        names.removeFirst();
        names.remove("Suman");

        // Queue operations
        names.offer("Extra");
        System.out.println("Peek: " + names.peek());

        // Iteration
        for (String n : names) {
            System.out.println(n);
        }
    }
}
```

5. Comparison: ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Underlying DS	Dynamic Array	Doubly Linked List
Access (get/set)	Fast (O(1))	Slow (O(n))
Insert/Delete (middle)	Slow (O(n))	Fast (O(1)) if iterator known
Insert/Delete (end)	Fast	Fast
Memory Usage	Less	More (extra node pointers)

Summary:

- **LinkedList** = good for **frequent insertions/deletions**.
- Implements **List + Deque** → works as **list + queue + stack**.
- Key methods: `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`, `peek()`, `poll()`.

ArrayDeque in Java

- **ArrayDeque** = **Resizable array-based implementation of Deque** (Double Ended Queue).
- Belongs to **`java.util` package**.
- Faster than `Stack` and `LinkedList` for **stack** and **queue** operations.
- **No capacity restriction** (grows dynamically).
- **Null elements not allowed**.
- Can function as both:
    - **Queue (FIFO)**
    - **Stack (LIFO)**

Declaration:

```
ArrayDeque<Type> dq = new ArrayDeque<>();
```

2. Constructors

`ArrayDeque()` → Creates an empty deque.
`ArrayDeque(int numElements)` → Creates deque with specified capacity.
`ArrayDeque(Collection c)` → Creates deque containing elements of another collection.

3. Commonly Used Methods

Insertion

- `addFirst(E e)` → Adds element at front.
- `addLast(E e)` → Adds element at rear.
- `offerFirst(E e)` → Adds at front, returns `true/false`.
- `offerLast(E e)` → Adds at rear, returns `true/false`.

```
dq.addFirst("A");
dq.addLast("B");
dq.offerFirst("X");
dq.offerLast("Y");
```

Removal

- `removeFirst()` → Removes and returns first element (exception if empty).
- `removeLast()` → Removes and returns last element.
- `pollFirst()` → Removes and returns first element (null if empty).
- `pollLast()` → Removes and returns last element (null if empty).
- `pop()` → Removes first element (stack behavior).

```
dq.removeFirst();
dq.pollLast();
String val = dq.pop();
```

Access (Peek/Check)

- `getFirst()` → Returns first element (exception if empty).
- `getLast()` → Returns last element.
- `peekFirst()` → Returns first element (null if empty).
- `peekLast()` → Returns last element (null if empty).

```
System.out.println(dq.getFirst());
System.out.println(dq.peekLast());
```

Stack Operations (LIFO)

- `push(E e)` → Adds element at front.
- `pop()` → Removes and returns element from front.

```
dq.push("Item1");  // same as addFirst()
dq.pop();          // same as removeFirst()
```

Other

- `size()` → Returns number of elements.

- `isEmpty()` → Checks if deque is empty.
- `iterator()` → Forward iteration.
- `descendingIterator()` → Reverse iteration.
- `clear()` → Removes all elements.

Example Code:

```
import java.util.*;

public class ArrayDequeDemo {
    public static void main(String[] args) {
        ArrayDeque<String> dq = new ArrayDeque<>();

        // Insertion
        dq.add("Apple");
        dq.addFirst("Start");
        dq.addLast("End");

        // Access
        System.out.println("First: " + dq.peekFirst());
        System.out.println("Last: " + dq.peekLast());

        // Removal
        dq.removeFirst();
        dq.pollLast();

        // Stack operations
        dq.push("Banana");
        System.out.println("Popped: " + dq.pop());

        // Iteration
        for (String s : dq) {
            System.out.println(s);
        }
    }
}
```

5. Advantages of ArrayDeque

- **Faster** than `LinkedList` (cache-friendly, less overhead).
- **Better** than `Stack` (modern replacement).
- Can be used as **stack, queue, deque**.
- Dynamic resizing (no fixed capacity).

Summary:

- **ArrayDeque** = best general-purpose implementation of **Deque**.
- Works as **Stack + Queue**.
- Key methods: `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`, `peekFirst()`, `peekLast()`, `push()`, `pop()`.
- More efficient than `Stack` and `LinkedList`.

Hashing in Java

- Hashing is a technique used to **map data to a fixed-size value (hash code)** using a hash function.
- This hash code helps in **fast searching, insertion, and deletion** of elements.

Key Points

1. **Hash Function** → Converts the input (like a string or number) into an integer (hash code).
2. **Hash Table** → Stores key-value pairs using the hash code as the index.
3. **Collision** → When two different inputs produce the same hash code.
    - Handled by **chaining** (linked list at same index) or **open addressing**.
4. **Advantages of Hashing**
    - Very **fast lookups** (on average O(1)).
    - Useful in sets, maps, and caching.

HashSet in Java

- `HashSet` is a **collection class** in Java that implements the **Set interface**.
- It uses a **hash table (backed by HashMap)** for storage.
- It **does not allow duplicate elements**.

Features of HashSet

- **No duplicates** → Each element is unique.
- **Unordered** → Does not maintain insertion order.
- **Null allowed** → Only **one null element** is allowed.
- **Backed by HashMap** → Internally uses hashing for fast operations.

Exmaple

```
import java.util.*;

class HashSetExample {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>();

        // Adding elements
        set.add("Apple");
        set.add("Banana");
        set.add("Mango");
        set.add("Banana"); // Duplicate, will not be added

        // Display elements
        System.out.println(set);

        // Checking
        System.out.println("Contains Mango? " + set.contains("Mango"));

        // Removing
        set.remove("Apple");
        System.out.println("After removing Apple: " + set);
    }
}
```

Important Methods of HashSet

- `add(E e)` → Adds element.
- `remove(Object o)` → Removes element.
- `contains(Object o)` → Checks if element exists.
- `size()` → Returns number of elements.

- `clear()` → Removes all elements.
- `isEmpty()` → Checks if set is empty.
- `iterator()` → Returns an iterator for traversal.

Difference between HashSet and List

Aspect	HashSet	List (ArrayList/LinkedList)
Order	No order maintained	Maintains insertion order
Duplicates	Not allowed	Allowed
Nulls	At most 1 null allowed	Multiple nulls allowed
Performance	Fast lookup (O(1) average)	Slower lookup (O(n))

File Handling in Java (CRUD Operations)

Java provides the **`java.io`** package for file handling.
CRUD stands for **Create, Read, Update, Delete**.

1. Import Required Classes

```
import java.io.File;
import java.io.FileWriter;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;
```

2. Create a File

```
public class CreateFileExample {
    public static void main(String[] args) {
        try {
            File file = new File("example.txt");
            if (file.createNewFile()) {
                System.out.println("File created: " + file.getName());
            } else {
                System.out.println("File already exists.");
            }
        } catch (IOException e) {
            System.out.println("An error occurred.");
            e.printStackTrace();
        }
    }
}
```

3. Write to a File (Create/Update)

```
public class WriteFileExample {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("example.txt"); // overwrite
            writer.write("Hello Java File Handling\n");
            writer.write("CRUD Operations Example");
            writer.close();
            System.out.println("Successfully written to file.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Append Mode:

```
FileWriter writer = new FileWriter("example.txt", true); // append mode
```

4. Read a File

Use **FileReader** and **BufferedReader**.

```
public class ReadFileExample {
    public static void main(String[] args) {
        try {
            File file = new File("example.txt");
            BufferedReader br = new BufferedReader(new FileReader(file));

            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
            br.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

5. Update a File

Read the file, modify content, and write back.

```
import java.io.*;

public class UpdateFileExample {
    public static void main(String[] args) {
        try {
            File file = new File("example.txt");
            BufferedReader br = new BufferedReader(new FileReader(file));

            StringBuilder sb = new StringBuilder();
            String line;
            while ((line = br.readLine()) != null) {
                sb.append(line.replace("Java", "JAVA")).append("\n");
            }
            br.close();

            FileWriter writer = new FileWriter(file);
            writer.write(sb.toString());
            writer.close();
            System.out.println("File updated successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

6. Delete a File

```
public class DeleteFileExample {
    public static void main(String[] args) {
        File file = new File("example.txt");
        if (file.delete()) {
            System.out.println("Deleted the file: " + file.getName());
        } else {
            System.out.println("Failed to delete the file.");
        }
```

```
    }
}
```

7. Key Points

File class is used to represent files/folders.
FileWriter → Writing text to file.
FileReader & BufferedReader → Reading text.
Updating → Read → Modify → Write.
Deletion → `file.delete()`.
Exception Handling is necessary (`IOException`).